

GABRIEL POPA

Senior Software Engineer - Angular • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | <https://www.linkedin.com/in/gabriel-popa-12890b414/>

ABOUT ME

Senior Software Engineer with **11+ years** of experience building **Angular frontend applications** for operational platforms, customer portals, analytics products, and administration systems across SaaS and business web environments. Strong specialization in **Angular 2–18, TypeScript 2.x–5.x, RxJS 5.x–7.x, NgRx, Angular Material, SCSS, and SPA architecture**, with consistent success modernizing legacy UI layers, stabilizing complex screen behavior, improving rendering performance, resolving production issues, and translating unclear product requirements into maintainable, reusable frontend systems. Background also includes practical collaboration with Node.js APIs, authentication flows, and cloud delivery, making cross-team frontend ownership more efficient.

SKILLS

- **Frontend: Angular 2–18, TypeScript 2.x–5.x**, JavaScript ES6+, RxJS 5.x–7.x, NgRx, Angular CLI, Angular Material, Reactive Forms, Template-Driven Forms, standalone components, route guards, lazy loading, reusable component architecture, SPA development, HTML5, CSS3, SCSS, responsive UI, Webpack, Vite basics, client-side validation, accessibility-minded UI development
- **Backend & Integration:** Node.js, Express.js, NestJS, REST API integration, JWT authentication flows, OAuth2 flows, RBAC-aware UI, WebSockets, API versioning awareness, file upload handling, background-job aware frontend integration, OpenAPI/Swagger collaboration
- **Data & State:** NgRx state management, RxJS stream composition, pagination patterns, caching-aware UI behavior, local/session storage strategies, table virtualization concepts, query parameter synchronization, form state orchestration, error-state modeling
- **Cloud & DevOps:** AWS collaboration, Docker, GitHub Actions, GitLab CI, Nginx environment awareness, Linux basics, frontend build pipelines, artifact packaging, release coordination, environment configuration, CI validation support
- **Observability / Quality / Security:** Jest, Jasmine, Karma, Cypress, Playwright, unit testing, integration testing, end-to-end testing, structured logging collaboration, Sentry, OpenTelemetry basics, correlation IDs, OWASP-aware frontend practices, route protection, rate-limit aware UX handling, audit-sensitive UI flows

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania (Jan 2024 – May 2026)

- Led delivery of a multi-tenant business platform using **Angular 14–18, TypeScript**, and modular UI architecture, converting fragmented dashboard, reporting, and administration requirements into scalable frontend workflows with stronger reuse and cleaner ownership boundaries.
- Modernized large legacy screen groups by restructuring them into feature-based Angular areas with shared components, route guards, typed services, and clearer folder conventions, which reduced regression risk and improved delivery speed for new product requests by **29%**.
- Improved rendering speed on data-heavy pages by profiling change-detection pressure, reducing unnecessary template recomputation, and refining observable composition with **RxJS**, which made filtering and table refresh behavior noticeably smoother under real user load.
- Migrated older **NgModule**-centric screens into a more maintainable standalone and component-driven Angular structure, which reduced coupling between features and lowered upgrade friction for future framework changes across the frontend codebase.
- Implemented complex onboarding and administration journeys with **Reactive Forms**, reusable validators, centralized error rendering, and conditional field logic, which improved validation consistency and reduced repeated form-related defects reported by QA by **33%**.

- Stabilized a recurring production issue where live tenant status panels stopped reflecting real changes after brief disconnects by improving reconnection handling, replay logic, and UI refresh sequencing around **WebSocket**-driven views.
- Built a reusable internal pattern library of tables, filter panels, action toolbars, drawers, and confirmation flows, helping product teams ship new screens faster while keeping interaction behavior and visual structure more consistent across modules.
- Solved a difficult screen-freeze problem triggered by overly eager subscriptions and stale listener cleanup by tracing component lifecycles, tightening unsubscribe patterns, and reducing memory-heavy state retention during repeated navigation.
- Strengthened security-sensitive frontend behavior by implementing **JWT**-aware route protection, **OAuth2** login flow handling, role-aware rendering, and safer unauthorized-state transitions so administrative actions were harder to expose accidentally.
- Improved frontend resilience around asynchronous operations by introducing idempotent UI action guards, duplicate-click prevention, and clearer retry feedback on submission-heavy flows where repeated user actions had previously created confusing states.
- Partnered closely with backend engineers during a Node.js runtime modernization effort, adjusting frontend integration contracts and validation assumptions so API changes could roll out without disrupting user-facing workflows or breaking active screens.
- Expanded operational visibility by wiring sensitive user journeys into **Sentry** and telemetry-friendly logging conventions, giving engineers faster insight into tenant-specific failures and reducing mean time to isolate frontend issues by **24%**.

Software Engineer

Next Apps | Ghent, Belgium (Nov2020 – Nov 2023)

- Built customer-facing product workflows with **Angular**, **TypeScript**, and **Angular Material**, focusing on validation-heavy pages, reusable UI interaction patterns, and clearer state transitions for submission flows that required predictable behavior under frequent change.
- Created a shared component library of inputs, tables, dialogs, status banners, and feedback patterns, which reduced duplicated implementation across teams and made design-system level UI updates significantly easier to deliver at scale.
- Improved maintainability by replacing scattered template logic with cleaner **RxJS** stream composition and service-based state handling, which reduced side-effect bugs and made frontend data flow easier for the team to reason about during feature delivery.
- Migrated older form implementations into **Angular Reactive Forms** with centralized validation rules, reusable field wrappers, and standardized error messaging, which improved consistency across larger submission journeys and reduced front-end defects by **31%**.
- Simplified frontend build and release behavior by cleaning up heavy **Webpack** configuration, removing unstable plugin usage, and improving environment-driven build settings so local development and CI execution became more predictable for contributors.
- Modernized older UI modules by replacing tightly coupled screen logic with smaller container and presentational component patterns, which made change review easier and reduced the risk of breaking unrelated features during enhancement work.
- Developed API-aware frontend flows with safer request-state handling, graceful loading and retry patterns, and clearer empty-state behavior so downstream contract changes were easier to absorb without producing broken experiences for end users.
- Implemented caching-aware data-fetching strategies and smarter refresh boundaries on commonly accessed pages, reducing unnecessary repeated calls to hot endpoints and improving perceived responsiveness on frequently revisited workflows by **26%**.
- Resolved intermittent timeout-related UX failures that surfaced as broken submissions and partial screen loads by coordinating with backend teams on latency visibility, then improving loading states, request handling, and user feedback around slow operations.
- Fixed a hard-to-diagnose consistency issue caused by out-of-order asynchronous updates by introducing safer client-side sequencing and reconciliation logic, which prevented stale values from briefly overriding the most recent valid screen state.
- Improved quality confidence by expanding component tests, interaction-focused UI checks, and critical flow coverage in **Cypress**, giving the team earlier warning on regressions before releases reached production-facing environments.

Software Developer

Datamart | Stockholm, Sweden (*Feb 2016 – Sep 2020*)

- Built internal analytics and administration tools using early **Angular** patterns, JavaScript, HTML, CSS, and practical UI modules that helped business users navigate large datasets through filtering, pagination, reporting screens, and responsive table workflows.
- Supported a gradual frontend refactor from controller-heavy page logic into more maintainable component-oriented structures, which improved reuse across internal products while preserving behavior that long-running teams depended on every day.
- Optimized slow reporting screens by reducing unnecessary client payload rendering, tightening filtering behavior, and coordinating with backend teams on query-heavy views, helping export and dashboard experiences feel materially faster for end users by **27%**.
- Modernized older interfaces by replacing brittle page scripts with clearer Angular component patterns, which improved state visibility for developers and made repeated admin workflow updates easier to implement with less duplicated code.
- Resolved a difficult date-handling issue caused by mismatches between browser input behavior, API serialization, and backend storage assumptions by standardizing UI formatting rules and validation behavior across the full reporting workflow.
- Implemented progress-aware frontend handling for large exports and long-running report generation, replacing frustrating synchronous wait behavior with clearer status feedback that made heavy operations easier for users to understand and trust.
- Added automated UI and integration coverage around sensitive reporting and administration paths so common regressions could be detected earlier during release cycles instead of surfacing after deployment to active internal teams.
- Improved CI reliability for frontend changes by separating fast verification checks from heavier integration execution, which reduced noisy failures and gave developers quicker confidence on safe UI updates before merging.
- Introduced more traceable admin interaction patterns, confirmation states, and audit-sensitive screen behavior, helping support teams understand who changed what and reducing confusion around disputed record updates in shared workflows.
- Reduced frontend asset weight through code splitting, dependency cleanup, and better loading boundaries, which improved startup time on slower devices and made large internal screens feel more responsive during daily operational use.

Junior Software Developer

Berg Software | Timișoara, Romania (*Sep2014 – Mar 2016*)

- Built early web modules using **JavaScript**, HTML, CSS, server-rendered UI patterns, and lightweight frontend enhancements, helping deliver stable request flows and practical user interfaces for internal and client-facing business pages.
- Improved legacy pages by introducing more modular client-side code, clearer validation feedback, and safer form behavior, which reduced user input mistakes and repeated support follow-up for common submission issues by **27%**.
- Helped resolve a recurring logout problem that surfaced as unpredictable session loss by tracing browser behavior, reviewing cookie handling, and aligning frontend expectations with backend timeout behavior to improve login stability.
- Developed reusable page helpers and shared UI fragments that reduced copy-paste implementation, making small enhancement requests and visual fixes easier for the team to apply across similar screens without increasing maintenance burden.
- Improved page performance by enabling compression-friendly asset handling, removing blocking scripts, and tightening browser caching behavior, which reduced first-render delays on frequently used pages and improved everyday usability.
- Supported release preparation by maintaining setup notes, testing UI fixes across environments, and assisting with safer deployment checklists so frontend changes reached shared systems with fewer avoidable environment-related problems.
- Assisted with bug resolution by reproducing issues locally, reviewing logs with senior engineers, and applying targeted low-risk fixes rather than broad refactors, which improved fix quality in fragile parts of the application.

- Improved admin dashboard usability by refining table behavior, form clarity, and basic data presentation patterns, helping the application handle growing record volume with more understandable interaction flow for operational users.
- Used structured debugging habits with reproduction notes, browser inspection, and incremental patches, building a reliable foundation in frontend problem-solving while lowering the chance of introducing new regressions during urgent fixes.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERTIFICATIONS

JavaScript Developer Certification — 2022

- Demonstrated strong understanding of modern **JavaScript**, asynchronous behavior, DOM interaction, modular design, debugging, and maintainable client-side development practices for interactive web applications.

Front-End Web Developer Professional Certificate — 2022

- Validated practical capability in responsive UI development, accessibility-minded implementation, reusable component thinking, browser behavior awareness, and structured frontend problem solving.